

Okkultes Wissen und Wissensorganisation

Vorlesung und Quellenübung

Sarah Lang*

2026

Vorlesung 'Okkultes Wissen und Wissensorganisation von Alchemie bis AI'

Quellenübung 'Okkultes Wissen und digitale Methoden'

*Max Planck Institute for the History of Science (Berlin)

Gastprofessur Digital Humanities, Bergische Universität Wuppertal

Machine Learning und Künstliche Intelligenz

KI, Maschinelles Lernen und Generative KI

Künstliche Intelligenz (KI)

Oberbegriff für Systeme, die Aufgaben ausführen, die normalerweise menschliche Intelligenz erfordern, beispielsweise Wahrnehmung, Entscheidungsfindung oder Sprachverarbeitung.

Maschinelles Lernen (ML)

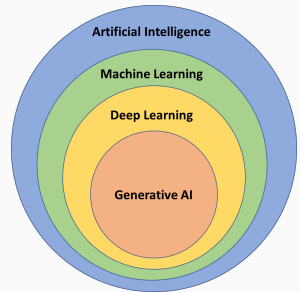
Teilgebiet der KI, bei dem Systeme aus Daten lernen und auf Basis von Mustern Vorhersagen oder Entscheidungen treffen.

Deep Learning

Teilgebiet des maschinellen Lernens, das mehrschichtige neuronale Netze verwendet und besonders große Datenmengen verarbeiten kann.

Generative KI

Modelle, die neue Inhalte wie Texte, Bilder, Audio, Code oder Videos erzeugen können.



Was ist Maschinelles Lernen?

Arthur Samuel (1959)

„Machine Learning ist das Forschungsgebiet, das Computern die Fähigkeit verleiht zu lernen, ohne explizit programmiert zu werden.“

Tom Mitchell (1998)

Ein Computerprogramm lernt aus einer Erfahrung **E** bezüglich einer Aufgabe **T**, wenn sich seine Leistung hinsichtlich eines Leistungsmaßes **P** durch diese Erfahrung verbessert.

Merksatz

Maschinelles Lernen bedeutet, aus Daten und 'Erfahrungen' Modelle zu erzeugen, die für neue Daten verallgemeinerbare Vorhersagen treffen können.

Aufgabe, Erfahrung und Leistungsmaß

Schachprogramm

- **Aufgabe (T):** Schach spielen
- **Erfahrung (E):** Tausende selbst gespielte Partien
- **Leistungsmaß (P):**
Gewinnwahrscheinlichkeit gegen neue Gegner

Spam-Erkennung

- **Aufgabe (T):** E-Mails klassifizieren
- **Erfahrung (E):** Markierungen als Spam oder Nicht-Spam
- **Leistungsmaß (P):** Anteil korrekt klassifizierter Nachrichten

Wie lernt ein Modell?

Grundprinzip

Ein Lernalgorithmus erhält Daten und sucht nach einer mathematischen Funktion, die die beobachteten Zusammenhänge möglichst gut beschreibt.

1. Trainingsdaten werden bereitgestellt.
2. Ein Modell wird an die Daten angepasst.
3. Die Qualität des Modells wird bewertet.
4. Das Modell wird optimiert.
5. Anschließend wird es auf neue Daten angewendet.

Modell

Das Ergebnis des Lernprozesses ist ein Modell, das Vorhersagen oder Klassifikationen für unbekannte Daten liefern kann.

Was ist Maschinelles Lernen?

Grundidee

Maschinelles Lernen (ML) beschäftigt sich mit Verfahren, die aus Daten und Erfahrungen lernen, anstatt für jede Aufgabe explizit programmiert zu werden.

Lernen aus Erfahrung

Ein Lernverfahren erhält Daten, erkennt Muster und erstellt daraus ein Modell. Dieses Modell kann anschließend Vorhersagen oder Entscheidungen für neue, bisher unbekannte Daten treffen.

Definition nach Tom Mitchell

Ein Computerprogramm lernt aus einer **Erfahrung E** bezüglich einer **Aufgabe T**, wenn sich seine Leistung hinsichtlich eines **Leistungsmaßes P** durch diese Erfahrung verbessert.

- **Aufgabe (T):** Was soll gelöst werden?
- **Erfahrung (E):** Welche Daten oder Beispiele stehen zur Verfügung?
- **Leistungsmaß (P):** Woran wird Erfolg gemessen?

Beispiele für Maschinelles Lernen

Schach- oder Damespiel

- **Aufgabe (T):** Das Spiel gewinnen.
- **Erfahrung (E):** Tausende selbst gespielte Partien.
- **Leistungsmaß (P):**
Gewinnwahrscheinlichkeit gegen neue Gegner.

Spam-Erkennung

- **Aufgabe (T):** E-Mails klassifizieren.
- **Erfahrung (E):** Vom Person als Spam oder Nicht-Spam markierte Nachrichten.
- **Leistungsmaß (P):** Anteil korrekt klassifizierter E-Mails.

Zentrale Idee

Die Leistung eines Systems verbessert sich mit zunehmender Erfahrung und zusätzlichen Daten.

Wie „lernt“ ein ML-Algorithmus?

Von Daten zum Modell

Klassische Programmierung definiert Regeln und erzeugt daraus Ergebnisse. Beim Maschinellen Lernen wird dieser Prozess umgekehrt:

- Eingabedaten werden bereitgestellt.
- Für viele Beispiele sind die gewünschten Ergebnisse bekannt.
- Der Algorithmus sucht selbst nach einer Funktion, die Eingaben auf Ausgaben abbildet.

Das Ergebnis: Ein Modell

Das gefundene Modell beschreibt Zusammenhänge in den Daten und kann anschließend auf neue, bisher unbekannte Beispiele angewendet werden.

Ziel

Nicht das Auswendiglernen der Trainingsdaten, sondern das Erkennen allgemeiner Muster.

Training und Generalisierung

Training

Während des Trainings passt der Lernalgorithmus seine Modellparameter so an, dass die Vorhersagen für die Trainingsdaten möglichst korrekt werden.

Generalisierung

Ein gutes Modell soll nicht nur auf den Trainingsdaten funktionieren, sondern auch auf neuen Daten zuverlässige Vorhersagen treffen.

Overfitting

Das Modell lernt die Trainingsdaten zu detailliert auswendig.

- Sehr gute Leistung auf Trainingsdaten
- Schlechte Leistung auf neuen Daten

Underfitting

Das Modell ist zu einfach und erfasst wichtige Zusammenhänge nicht.

- Schlechte Leistung auf Trainingsdaten
- Schlechte Leistung auf Testdaten

Die drei Bausteine des Lernprozesses

1. Repräsentation

- Festlegung, wie Wissen dargestellt wird.
- Der Algorithmus erstellt ein Modell, das Eingaben auf Ausgaben abbildet.
- Relevante Merkmale (*Features*) müssen erkannt oder ausgewählt werden.

2. Bewertung

- Verschiedene Modelle müssen miteinander verglichen werden.
- Eine Bewertungs- oder Fehlerfunktion misst die Qualität der Vorhersagen.

3. Optimierung

- Der Algorithmus verbessert das Modell schrittweise.
- Ziel ist die Minimierung des Fehlers bzw. die Maximierung der Modellgüte.

Trainingsdaten, Testdaten und Modellqualität

Trainingsdaten

Mit diesen Daten wird das Modell gelernt und angepasst.

Testdaten

Diese Daten werden erst nach dem Training verwendet und dienen zur unabhängigen Bewertung des Modells.

Genauigkeit (*Accuracy*)

Die Genauigkeit beschreibt, wie gut ein Modell auf bisher unbekannten Daten arbeitet.

- Die Auswahl geeigneter Trainingsdaten ist entscheidend.
- Trainings- und Testdaten müssen sauber getrennt werden.
- Schlechte oder verzerrte Daten führen häufig zu schlechten Modellen.

Fehlerfunktionen und Modellanpassung

Warum benötigt ein Algorithmus eine Fehlerfunktion?

Ein Lernalgorithmus kann viele unterschiedliche Modelle erzeugen. Um zwischen guten und schlechten Modellen zu unterscheiden, wird ein quantitatives Gütemaß benötigt.

Fehler- bzw. Kostenfunktion

Die Fehlerfunktion misst die Abweichung zwischen den tatsächlichen und den vorhergesagten Werten.

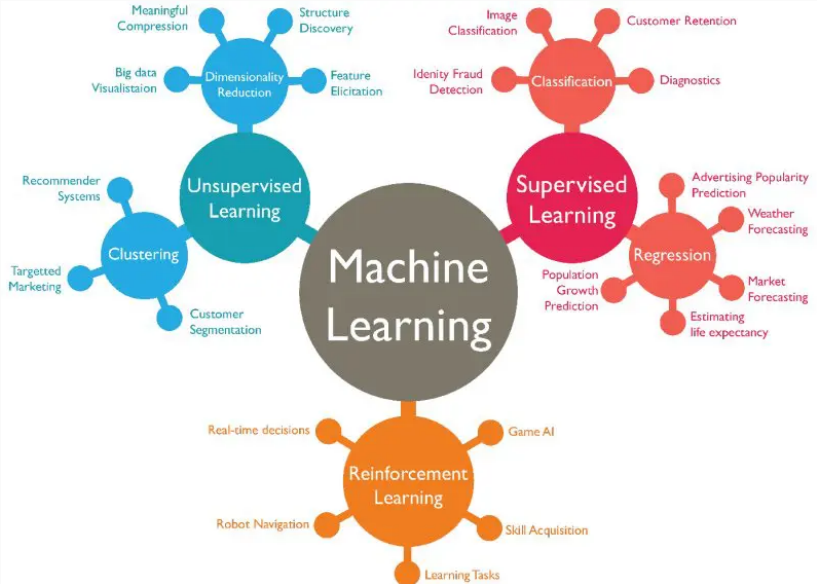
- Große Fehler sollen (stark) bestraft werden.
- Kleine Fehler sollen weniger bestraft werden.
- Das Training versucht, die Fehlerfunktion zu minimieren.

Merksatz

Maschinelles Lernen bedeutet letztlich, aus Daten ein Modell zu finden, das den Fehler auf neuen Daten möglichst klein hält.

Typen von ML

Arten des Maschinellen Lernens



Überwachtes Lernen (Supervised Learning)

Grundidee

Das Modell wird mit gelabelten Daten trainiert. Die Zielvariable ist bekannt und soll für neue Daten vorhergesagt werden.

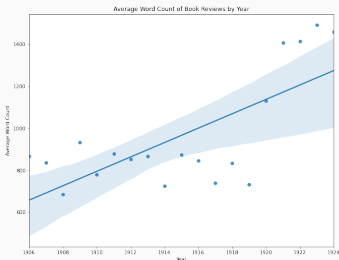
Klassifikation

- Vorhersage diskreter Klassen
- Beispiele:
 - Spam / Nicht-Spam
 - Positiv / Negativ
 - Dokumentgattungen
- Typische Verfahren:
 - Logistische Regression
 - SVM
 - Naive Bayes

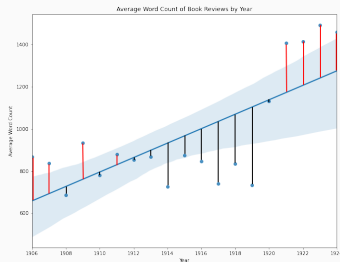
Regression

- Vorhersage kontinuierlicher Werte
- Beispiele:
 - Preise
 - Temperaturen
 - Textlängen
- Typische Verfahren:
 - Lineare Regression
 - Entscheidungsbäume
 - Random Forests

Beispiel: Lineare Regression



Eine Gerade wird so angepasst, dass sie den Zusammenhang der Daten möglichst gut beschreibt.



Die Abweichung zwischen Vorhersage und tatsächlichem Wert wird durch eine Fehlerfunktion gemessen.

Optimierung

Das Training versucht, den Fehler zu minimieren und dadurch ein möglichst gutes Modell zu erzeugen.

Unüberwachtes Lernen (Unsupervised Learning)

Grundidee

Die Daten besitzen keine bekannten Zielwerte. Das Ziel besteht darin, Strukturen, Gruppen oder Muster in den Daten zu entdecken.

Clustering

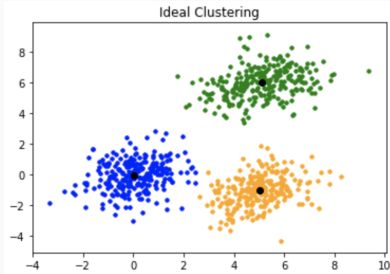
- Bildung ähnlicher Gruppen
- Beispiele:
 - Dokumentgruppen
 - Bildsammlungen
 - Handschriften
- Verfahren: K-Means

Dimensionsreduktion

- Reduktion vieler Merkmale auf wenige Dimensionen
- Visualisierung komplexer Daten
- Verfahren: PCA

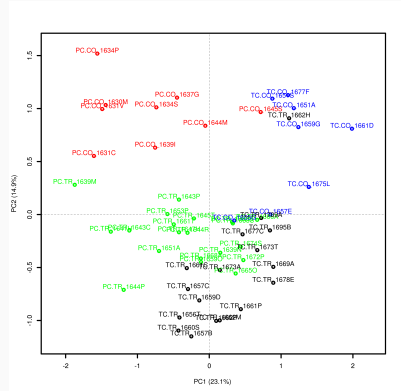
Beispiele: K-Means und PCA

K-Means



Bildet automatisch Gruppen ähnlicher Objekte.

PCA



Reduziert komplexe Daten auf wenige Dimensionen und macht Strukturen sichtbar.

Bestärkendes Lernen (Reinforcement Learning)

Grundidee

Ein Agent lernt durch Interaktion mit einer Umgebung. Für erfolgreiche Aktionen erhält er Belohnungen, für unerwünschte Aktionen Strafen.

- Lernen durch Trial-and-Error
- Ziel ist die Maximierung langfristiger Belohnungen
- Anwendungen:
 - Spiele (Schach, Go)
 - Robotik
 - Autonome Systeme

Von Deep Learning zu Generativer KI

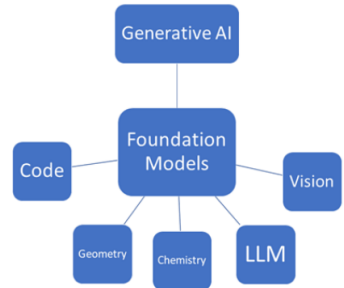
Deep Learning

Verwendet mehrschichtige neuronale Netze, um komplexe Muster in großen Datenmengen zu lernen.

Generative KI

Generiert neue Inhalte auf Basis gelernter Wahrscheinlichkeiten und Muster.

- Texte
- Bilder
- Audio
- Video
- Programmcode



Large Language Models (LLMs)

Was sind LLMs?

Large Language Models sind neuronale Netze, die auf sehr großen Textmengen trainiert werden und natürliche Sprache verarbeiten können.

- Textgenerierung
- Zusammenfassungen
- Übersetzungen
- Informationsgewinnung
- Klassifikation von Texten

“Attention Is All You Need”

Moderne LLMs basieren auf der Transformer-Architektur und lernen statistische Beziehungen zwischen Wörtern und Textbausteinen.

Trainingsdaten und
'Ground Truth' am Bsp.
OCR/HTR

Trainingsdaten, Testdaten und Modellqualität

Trainingsdaten

Dienen zum Lernen der Modellparameter.

Testdaten

Dienen zur unabhängigen Bewertung des Modells.

Overfitting

Das Modell merkt sich die Trainingsdaten und generalisiert schlecht.

Underfitting

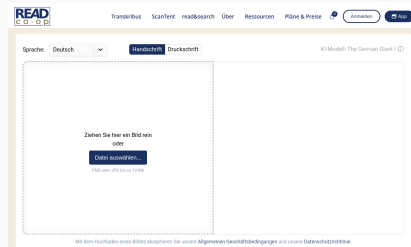
Das Modell ist zu einfach und erkennt wichtige Muster nicht.

Ziel

Ein Modell soll nicht nur bekannte Daten korrekt verarbeiten, sondern auch auf unbekannte Daten zuverlässig reagieren.

Ein Beispiel aus den Digital Humanities: OCR und HTR

- OCR (*Optical Character Recognition*) erkennt gedruckten Text in Bildern.
- HTR (*Handwritten Text Recognition*) erkennt handschriftliche Texte oder historische Drucke.
- Moderne Systeme nutzen Deep Learning und große Trainingsdatensätze.
- Anwendungen:
 - Digitalisierung historischer Quellen
 - Volltextsuche
 - Automatische Transkription



OCR/HTR als praktisches Beispiel für ML

Motivation

Viele gedruckte und handschriftliche Quellen sind bereits gescannt und online verfügbar. Als reine Bilddateien sind sie jedoch nur eingeschränkt nutzbar.

Ziel

Durch OCR und HTR entstehen maschinenverarbeitbare Texte, die durchsucht, annotiert, korrigiert und quantitativ analysiert werden können.

- **OCR:** Texterkennung für gedruckte Quellen.
- **HTR:** Texterkennung für handschriftliche oder historische Quellen.
- **ML-Perspektive:** Ein Modell lernt aus Beispielen, wie Bildinformationen in Text übertragen werden.

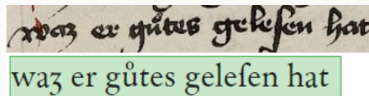
Von der Quelle zum maschinenlesbaren Text

Grundidee

OCR/HTR wandelt Bilder von Text in digitalen Text um. Moderne Verfahren erkennen dabei häufig nicht einzelne Zeichen, sondern ganze Textzeilen.

Typischer Ablauf

1. Scans oder Digitalisate vorbereiten.
2. Layout und Textregionen erkennen.
3. Zeilen segmentieren.
4. Zeilenbilder in Text umwandeln.
5. Ergebnis korrigieren und evaluieren.



Zeilenbild-Text-Paar als zentrale Trainingseinheit.

Hauptkomponenten

- **Vorverarbeitung:** Verbesserung der Bildqualität.
- **Segmentierung:** Erkennung von Regionen, Zeilen und Textfluss.
- **Texterkennung:** Umwandlung von Bildinformationen in Text.
- **Nachkorrektur:** Manuelle oder halbautomatische Verbesserung.
- **Evaluation:** Messung der Ergebnisqualität.



Segmentierung: Text muss zuerst gefunden werden

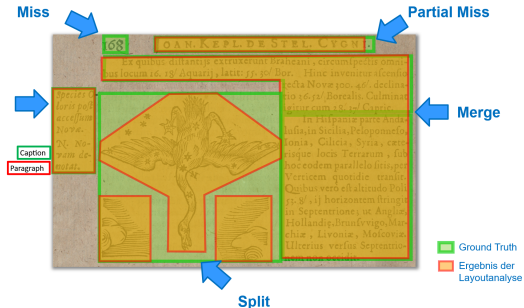
Was passiert bei der Segmentierung?

Eine Seite wird in relevante Bereiche unterteilt: Textblöcke, Überschriften, Randnotizen, Tabellen, Zeilen oder andere Layoutbestandteile.

Warum ist das schwierig?

Historische Quellen enthalten oft komplexe Layouts, schiefe Zeilen, beschädigte Seiten, Mehrspaltigkeit, Marginalien oder wechselnde Schriftgrößen.

Fehler in der Segmentierung führen häufig zu Fehlern in der Texterkennung.



Training: Wie OCR/HTR-Modelle lernen

Trainingsdaten

Ein OCR/HTR-Modell lernt anhand von Paaren aus Bild und korrektem Text. Häufig sind dies Zeilenbilder mit der entsprechenden Transkription.

Input

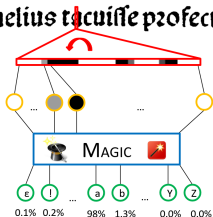
- Bild einer Textzeile
- visuelle Merkmale der Schrift
- Abstände, Zeichenformen, Ligaturen

Output

- transkribierte Zeichenfolge
- möglichst nahe an der Ground Truth
- anwendbar auf neue Zeilenbilder

Texterkennung - zeilenbasiert

melius tunc uisite profecto



Seht nur!

Seht nur!

SSS-eee-lhh-t---mnn-nuu-rrr-!!!

Seht nur!

Modelle: allgemein oder spezialisiert?

Gemischte Modelle

Gemischte Modelle werden auf vielen unterschiedlichen Quellen trainiert. Sie sind schnell einsetzbar, aber für schwierige historische Quellen oft weniger genau.

Spezifische Modelle

Werkspezifische oder quellenspezifische Modelle werden auf eine bestimmte Schrift, einen bestimmten Drucktyp oder eine bestimmte Sammlung angepasst.

- Spezifische Modelle benötigen Ground Truth.
- Sie sind aufwändiger in der Vorbereitung.
- Sie liefern bei historischen oder ungewöhnlichen Quellen häufig bessere Ergebnisse.

Ground Truth: Referenz für Training und Evaluation

Definition

Ground Truth bezeichnet eine möglichst korrekte Wiedergabe der Vorlage. Sie dient als Referenz, an der automatische Ergebnisse gemessen werden.

Warum ist Ground Truth wichtig?

- Sie ermöglicht das Training von Modellen.
- Sie erlaubt die Messung der Ergebnisqualität.
- Sie macht Fehler systematisch sichtbar.

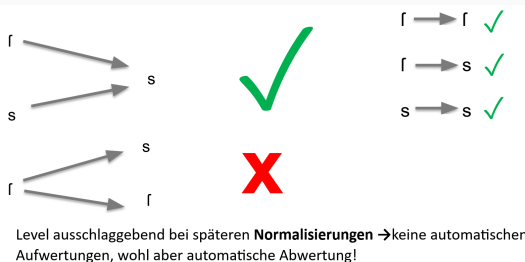
Aber Achtung!

Ground Truth ist nie völlig neutral: Transkriptionsregeln bestimmen, was als „korrekt“ gilt.

Ground Truth: Transkriptionsregeln

Mögliche Normalisierung

- historische Schreibweisen beibehalten oder modernisieren
- langes s als eigenes Zeichen oder als normales s erfassen
- Abkürzungen auflösen oder originalgetreu übernehmen
- Groß- und Kleinschreibung exakt oder normalisiert erfassen



Der gewählte Transkriptionslevel beeinflusst sowohl das Modelltraining als auch spätere Fehlerquoten.

Ground Truth erstellen

Traditioneller Ansatz

Ground Truth wird manuell transkribiert, ohne vorherige automatische Texterkennung. Das ist präzise, aber zeitaufwändig.

Iterativer Ansatz

Ein vorhandenes Modell erzeugt eine Vortranskription. Diese wird korrigiert und anschließend als neue Trainingsgrundlage verwendet.

Mögliche Kontexte

- **Forschungsprojekt:** hohe Qualität durch Fachpersonal.
- **Citizen Science:** große Datenmengen, aber variable Qualität.
- **Lehre:** didaktisch sinnvoll, Qualität abhängig von Anleitung und Kontrolle.

Evaluation: Wie gut ist ein OCR/HTR-Ergebnis?

Grundfrage

Wie stark weicht das automatisch erkannte Ergebnis von der Ground Truth ab?

Fehlerarten

- **Insertion:** Ein Zeichen wurde zusätzlich eingefügt.
- **Deletion:** Ein Zeichen fehlt.
- **Substitution:** Ein Zeichen wurde durch ein anderes ersetzt.

Typische Metriken

- **CER:** Character Error Rate
- **WER:** Word Error Rate
- **Confusion Matrix:** systematische Verwechslungen

Character Error Rate (CER)

Definition

Die Character Error Rate misst, wie viele Zeichenoperationen nötig sind, um das OCR/HTR-Ergebnis in die Ground Truth zu überführen.

$$\text{CER} = \frac{I + S + D}{N} \times 100$$

- **I:** Insertions
- **S:** Substitutions
- **D:** Deletions
- **N:** Anzahl der Zeichen in der Ground Truth

Interpretation

Je niedriger die CER, desto besser ist das Ergebnis. Eine CER von 0 bedeutet eine perfekte Übereinstimmung.

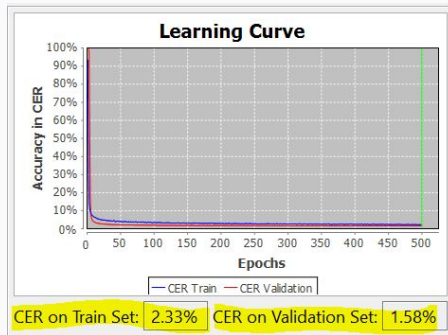
CER und Levenshtein-Distanz

Levenshtein-Distanz

Die Levenshtein-Distanz ist die minimale Anzahl von Einfügungen, Löschungen und Ersetzungen, die nötig ist, um eine Zeichenkette in eine andere umzuwandeln.

Bedeutung für OCR/HTR

CER verwendet dieses Prinzip auf Zeichenebene. Auch kleine Abweichungen wie fehlende Kommata, zusätzliche Leerzeichen oder falsche Großschreibung zählen als Fehler.



Word Error Rate (WER)

Definition

Die Word Error Rate misst Fehler auf Wortebene. Ein Wort gilt als fehlerhaft, wenn es nicht exakt mit der Ground Truth übereinstimmt.

$$\text{WER} = \frac{\text{fehlerhafte Wörter}}{\text{Wörter in der Ground Truth}}$$

- WER ist oft strenger wahrnehmbar als CER.
- Ein einzelner Zeichenfehler kann ein ganzes Wort fehlerhaft machen.
- WER eignet sich besonders, wenn die Lesbarkeit oder Suchbarkeit ganzer Wörter relevant ist.

Übung: CER und WER

Gegeben sind:

Ground Truth: Wir essen, mein Kind

OCR-Ergebnis: Wir essen meijn Kind

Welche Fehler liegen vor?

Wie hoch sind CER und WER?

Metriken: Rechenbeispiel

Gegeben

- GT: *Wir essen, mein Kind*
- OCR: *Wir essen meijn Kind*

Lösung

- CER: $2/20 = 0,1 = 10\%$
- WER: $1/4 = 0,25 = 25\%$

	W i r e s s e n , m e i n K i n d																				
	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
W i r	1	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
	2	1	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
	3	2	1	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
	4	3	2	1	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
e s s	5	4	3	2	1	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
	6	5	4	3	2	1	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
	7	6	5	4	3	2	1	0	1	2	3	4	5	6	7	8	9	10	11	12	13
	8	7	6	5	4	3	2	1	0	1	2	3	4	5	6	7	8	9	10	11	12
n	9	8	7	6	5	4	3	2	1	0	1	2	3	4	5	6	7	8	9	10	11
	10	9	8	7	6	5	4	3	2	1	1	1	2	3	4	5	6	7	8	9	10
m e	11	10	9	8	7	6	5	4	3	2	2	2	1	2	3	4	5	6	7	8	9
	12	11	10	9	8	7	6	5	4	3	3	3	2	1	2	3	4	5	6	7	8
i j	13	12	11	10	9	8	7	6	5	4	4	4	3	2	1	2	3	4	5	6	7
	14	13	12	11	10	9	8	7	6	5	5	5	4	3	2	2	3	4	5	6	7
n	15	14	13	12	11	10	9	8	7	6	6	6	5	4	3	2	3	4	5	5	6
	16	15	14	13	12	11	10	9	8	7	7	6	6	5	4	3	2	3	4	5	6
K i	17	16	15	14	13	12	11	10	9	8	8	7	7	6	5	4	3	2	3	4	5
	18	17	16	15	14	13	12	11	10	9	9	8	8	7	6	5	4	3	2	3	4
n d	19	18	17	16	15	14	13	12	11	10	10	9	9	8	7	6	5	4	3	2	3
	20	19	18	17	16	15	14	13	12	11	11	10	10	9	8	7	6	5	4	3	2

Was lernen wir daraus über Maschinelles Lernen?

OCR/HTR als Beispiel für ML

OCR/HTR zeigt zentrale Konzepte des maschinellen Lernens in einer für die Geisteswissenschaften vertrauten Anwendung.

- **Trainingsdaten:** Zeilenbilder mit korrekter Transkription.
- **Ground Truth:** Referenz für Lernen und Bewertung.
- **Modell:** gelernte Abbildung von Bild zu Text.
- **Evaluation:** Messung der Abweichung mit CER oder WER.
- **Generalisierung:** Anwendung auf neue, unbekannte Quellen.

Wichtig:

Die Qualität eines ML-Systems hängt nicht nur vom Algorithmus ab, sondern auch von Daten, Regeln, Vorverarbeitung und Evaluation.

Einführung in neuronale Netze nach 3Blue1Brown

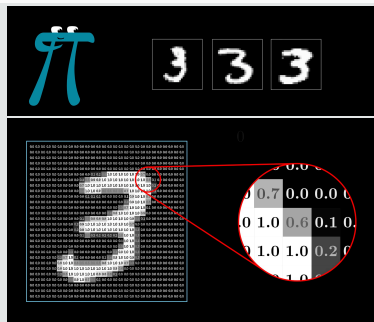
Wie lernen neuronale Netze? Ein einfaches Beispiel

Handschriftliche Ziffernerkennung

Ein klassisches Einstiegsbeispiel für neuronale Netze ist die Erkennung handgeschriebener Ziffern von 0 bis 9.

- **Input:** ein Bild mit 28×28 Pixeln.
- **Output:** zehn mögliche Klassen, also die Ziffern 0 bis 9.
- **Aufgabe:** Das Netz soll aus Beispielen lernen, welche Pixelmuster zu welcher Ziffer gehören.
- **Problem:** Menschen erkennen Ziffern leicht, aber explizite Regeln dafür zu schreiben ist schwierig.

Das Netzwerk erhält keine festen Regeln wie „wenn diese Pixel hell sind, dann ist es eine 3“. Stattdessen wird es mit vielen gelabelten Beispielen trainiert und passt seine internen Parameter an.



Beispiele handgeschriebener Ziffern.

Diese und folgende Slides (und Bilder) nach Sanderson und Pullen 2017a; Sanderson und Pullen 2017b

Aufbau eines einfachen neuronalen Netzes

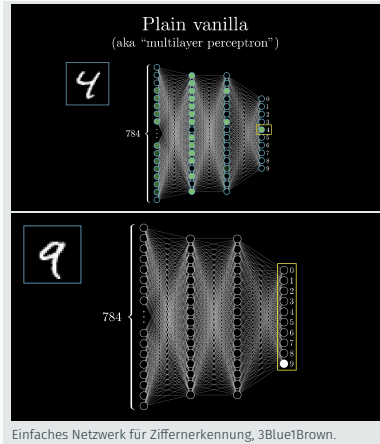
Schichten

Ein einfaches neuronales Netz besteht aus mehreren Schichten von Neuronen. Ein Neuron kann man zunächst als eine Einheit verstehen, die eine Zahl zwischen 0 und 1 speichert.

- **Input Layer:** 784 Neuronen für 28×28 Pixel.
- **Hidden Layers:** Zwischenschichten, in denen Muster verarbeitet werden.
- **Output Layer:** 10 Neuronen für die Ziffern 0 bis 9.

Aktivierung

Die Zahl in einem Neuron heißt **Aktivierung**. Hohe Aktivierung bedeutet, dass das Neuron für den aktuellen Input stark „anspricht“.



Warum mehrere Schichten?

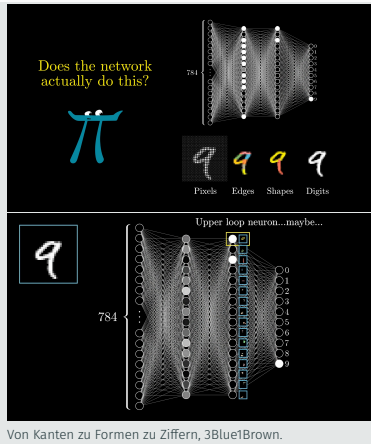
Probleme in kleinere Schritte zerlegen

Die Hoffnung hinter der Schichtenstruktur ist, dass komplexe Aufgaben in einfachere Teilaufgaben zerlegt werden.

- Frühe Schichten können einfache Muster wie Kanten erkennen.
- Mittlere Schichten können daraus Linien, Schleifen oder Bögen zusammensetzen.
- Spätere Schichten können Kombinationen dieser Muster bestimmten Ziffern zuordnen.

Beispiel

Eine „8“ kann als Kombination aus zwei Schleifen verstanden werden. Eine „1“ besteht eher aus einer langen Linie. Das Netz muss solche Merkmale nicht manuell vorgegeben bekommen, sondern soll sie aus Trainingsdaten lernen.



Gewichte: Verbindungen zwischen Neuronen

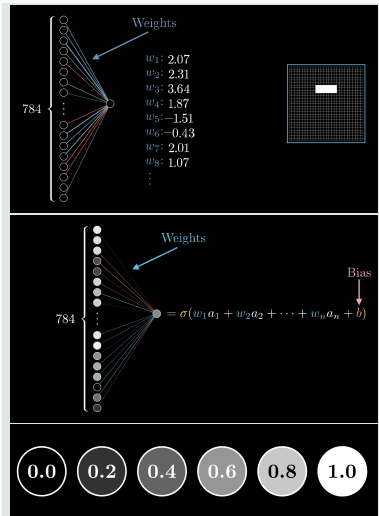
Gewichtete Verbindungen

Jedes Neuron einer Schicht ist mit Neuronen der nächsten Schicht verbunden. Diese Verbindungen haben **Gewichte**.

- Positive Gewichte verstärken eine Aktivierung.
- Negative Gewichte schwächen eine Aktivierung.
- Die gewichtete Summe bestimmt, wie stark ein Neuron in der nächsten Schicht aktiviert wird.

Intuition

Ein Neuron kann beispielsweise lernen, auf ein bestimmtes Kantenmuster zu reagieren: Helle Pixel an einer Stelle erhöhen die Aktivierung, störende Pixel in der Umgebung verringern sie.



Weights & Biases, 3Blue1Brown.

Bias und Aktivierungsfunktion

Bias

Zusätzlich zu den Gewichten besitzt jedes Neuron einen **Bias**. Er verschiebt die Schwelle, ab der ein Neuron stark aktiviert wird.

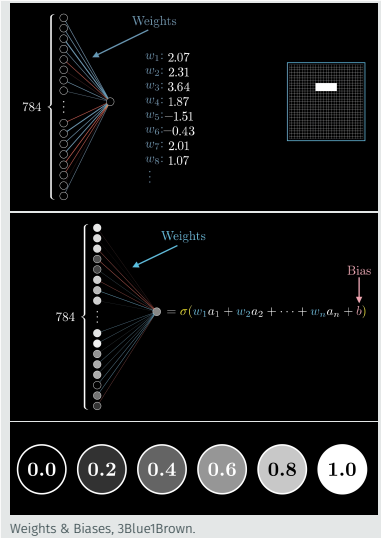
- Gewichte bestimmen, auf welches Muster ein Neuron reagiert.
- Der Bias bestimmt, wie deutlich dieses Muster vorhanden sein muss.

Aktivierungsfunktion

Die gewichtete Summe kann sehr groß oder sehr klein sein. Eine Aktivierungsfunktion bringt diesen Wert in einen kontrollierten Bereich, zum Beispiel zwischen 0 und 1.

z.B. Sigmoid-Funktion

$$a = \sigma(w_1x_1 + w_2x_2 + \dots + w_nx_n + b)$$



Das Netzwerk ist eine Funktion

Von 784 Zahlen zu 10 Zahlen

Ein neuronales Netz kann als große Funktion verstanden werden:

$$f: \mathbb{R}^{784} \rightarrow \mathbb{R}^{10}$$

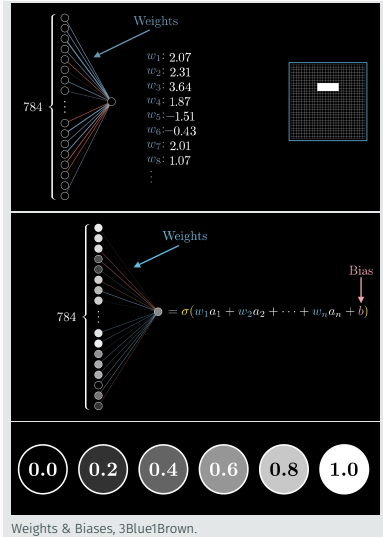
Es nimmt die Pixelwerte eines Bildes auf und gibt zehn Aktivierungen aus, eine für jede mögliche Ziffer.

Parameter

Die Funktion wird durch viele Gewichte und Bias-Werte bestimmt. In dem einfachen Beispielnetz sind es bereits über 13.000 veränderbare Parameter.

Lernen

Lernen bedeutet, diese Parameter so anzupassen, dass die Ausgaben des Netzes möglichst gut zu den gelabelten Trainingsdaten passen.



Training: Fehler messen und Parameter anpassen

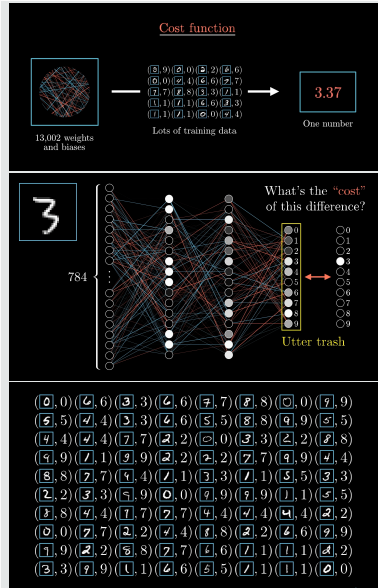
Trainingsbeispiel

Für jedes Trainingsbild ist bekannt, welche Ziffer es zeigt. Das Netz erzeugt eine Vorhersage, die mit diesem korrekten Label verglichen wird.

- Ist die Vorhersage richtig, ist der Fehler klein.
- Ist die Vorhersage falsch, ist der Fehler groß.
- Eine Fehlerfunktion misst die Abweichung zwischen Output und gewünschtem Output.

Optimierung

Das Training verändert Gewichte und Bias-Werte schrittweise so, dass der Fehler über viele Trainingsbeispiele kleiner wird.



Gradient Descent: Lernen als schrittweise Verbesserung

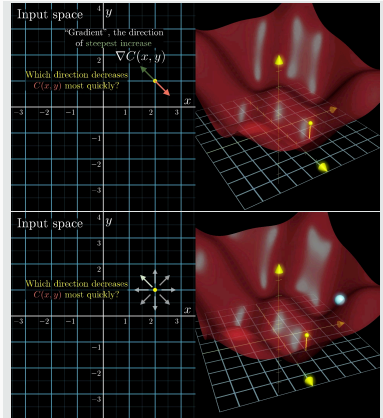
Grundidee

Gradient Descent ist ein Verfahren, mit dem die Parameter eines Modells so verändert werden, dass die Fehlerfunktion kleiner wird.

- Die aktuelle Einstellung der Gewichte erzeugt einen bestimmten Fehler.
- Der Gradient zeigt, in welche Richtung der Fehler am stärksten steigt.
- Das Modell geht einen kleinen Schritt in die entgegengesetzte Richtung.
- Dieser Vorgang wird viele Male mit vielen Beispielen wiederholt.

Intuition

Das Modell „lernt“ nicht durch Verständnis, sondern durch wiederholte kleine Korrekturen an seinen Parametern.



Fehlerlandschaft und Gradient Descent, 3Blue1Brown.

Backpropagation: Welcher Parameter war verantwortlich?

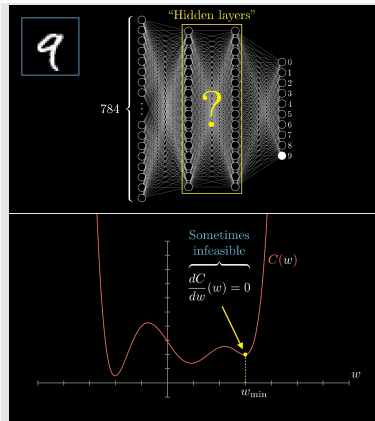
Problem

Ein neuronales Netz besitzt viele Gewichte und Bias-Werte. Wenn der Output falsch ist, muss bestimmt werden, welche Parameter wie stark zur Abweichung beigetragen haben.

Backpropagation

Backpropagation berechnet den Einfluss einzelner Parameter auf den Fehler, Schicht für Schicht rückwärts vom Output zum Input.

- Der Fehler wird am Output gemessen.
- Danach wird er rückwärts durch das Netzwerk verteilt.
- So erhält jeder Parameter eine Richtung für die Korrektur.



Suche/Lösten des Problems durch Hidden Layers und Problem bei der Suche von Minima, 3Blue1Brown.

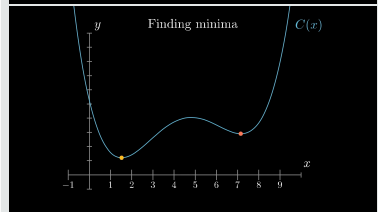
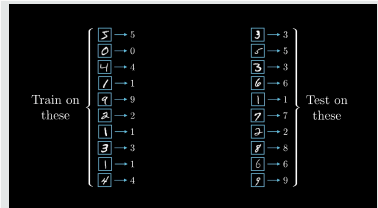
Was lernen wir daraus?

Neuronale Netze in einem Satz

Ein neuronales Netz ist eine parametrisierte Funktion, die Eingaben in Ausgaben übersetzt und ihre Parameter anhand von Trainingsdaten verbessert.

Zentrale Begriffe

- **Aktivierung:** Zahlenwert eines Neurons.
- **Gewicht:** Stärke einer Verbindung zwischen Neuronen.
- **Bias:** Schwellenwert für Aktivierung.
- **Fehlerfunktion:** Maß für falsche Vorhersagen.
- **Gradient Descent:** schrittweise Verringerung des Fehlers.
- **Backpropagation:** Berechnung der Parameterkorrekturen.



Trainingsschleife: Vorhersage, Fehler, Korrektur.

Gradient Descent: Wie neuronale Netze lernen

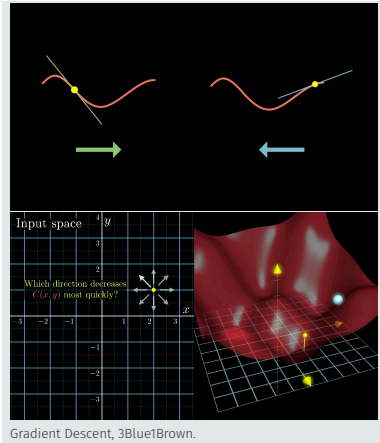
Ausgangspunkt

Ein neuronales Netz beginnt meist mit zufälligen Gewichten und Bias-Werten. Zu Beginn sind seine Vorhersagen daher schlecht.

- Ein Bild mit 28×28 Pixeln ergibt 784 Input-Werte.
- Das Netz gibt 10 Werte aus, einen für jede Ziffer.
- Die höchste Aktivierung im Output Layer ist die Vorhersage des Netzes.

Lernen

Das Netz lernt, indem es seine Gewichte und Bias-Werte so verändert, dass seine Vorhersagen besser zu den gelabelten Trainingsdaten passen.



Die Cost Function: Fehler messbar machen

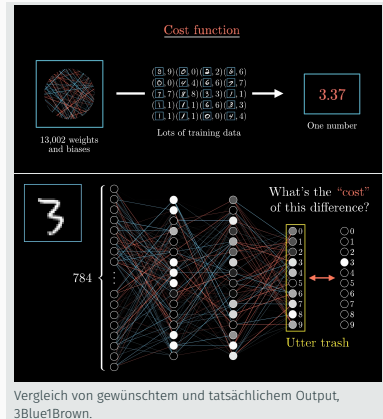
Was misst die Cost Function?

Die Cost Function misst, wie stark die Vorhersage des Netzwerks vom korrekten Label abweicht.

- Für ein Bild einer „3“ sollte das Output-Neuron für 3 nahe bei 1 liegen.
- Alle anderen Output-Neuronen sollten nahe bei 0 liegen.
- Je stärker die Abweichung, desto höher der Cost-Wert.

Über viele Beispiele

Im Training interessiert nicht nur der Fehler eines einzelnen Bildes, sondern der durchschnittliche Fehler über viele Trainingsbeispiele.



Lernen als Minimierung des Fehlers

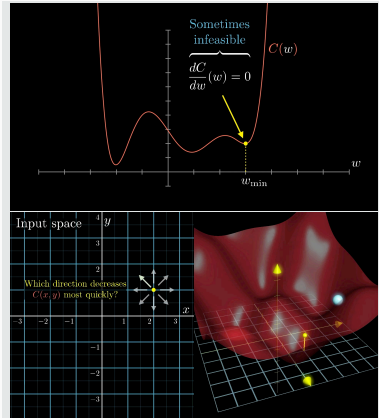
Mathematische Idee

Die Cost Function erhält alle Gewichte und Bias-Werte als Eingabe und gibt eine Zahl aus: den Fehler des Netzwerks.

Ziel

Training bedeutet, Werte für Gewichte und Biases zu finden, bei denen die Cost Function möglichst klein ist.

- Kleine Cost-Werte bedeuten gute Vorhersagen.
- Große Cost-Werte bedeuten schlechte Vorhersagen.
- Lernen ist daher ein Optimierungsproblem.



Fehlerlandschaft mit lokalen Minima, 3Blue1Brown.

Gradient Descent: Schrittweise bergab

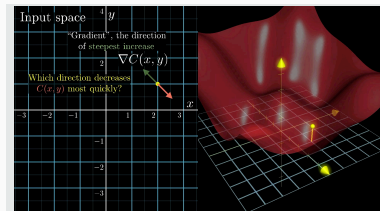
Intuition

Man kann sich die Cost Function wie eine Landschaft vorstellen. Das Ziel ist ein möglichst tiefer Punkt in dieser Landschaft.

- Der Gradient zeigt in die Richtung des stärksten Anstiegs.
- Der negative Gradient zeigt in die Richtung des stärksten Abstiegs.
- Das Modell macht viele kleine Schritte in diese Abstiegsrichtung.

Learning Rate

Die Learning Rate bestimmt die Schrittgröße. Zu große Schritte können das Minimum verfehlen, zu kleine Schritte machen das Training langsam.



Gradient Descent als Bewegung bergab, 3Blue1Brown.

Der Gradient als Korrekturanweisung

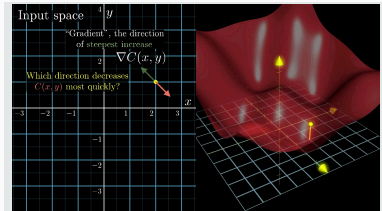
Viele Parameter

Ein einfaches Netz zur Ziffernerkennung kann bereits über 13.000 Gewichte und Bias-Werte besitzen.

Was sagt der Gradient?

Der negative Gradient enthält für jeden Parameter eine kleine Korrekturanweisung.

- Das Vorzeichen sagt, ob ein Parameter erhöht oder verringert werden soll.
- Die Größe sagt, wie wichtig diese Änderung für die Senkung des Fehlers ist.
- Parameter mit größerem Einfluss werden stärker angepasst.



Negativer Gradient, 3Blue1Brown.

Was bedeutet „Lernen“ hier genau?

Kurzfassung

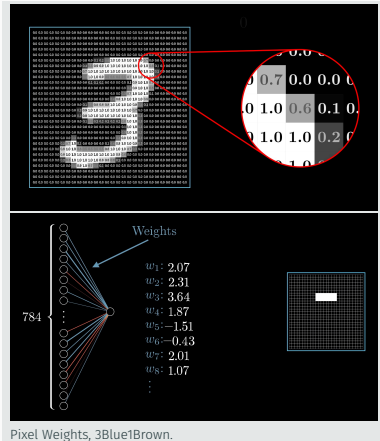
Ein neuronales Netz lernt, indem es Gewichte und Bias-Werte verändert, um eine Cost Function zu minimieren.

Trainingsschleife

1. Ein gelabeltes Trainingsbild wird eingegeben.
2. Das Netz berechnet eine Vorhersage.
3. Die Cost Function misst den Fehler.
4. Gradient Descent bestimmt sinnvolle Parameteränderungen.
5. Die Gewichte und Biases werden angepasst.

Merke

„Lernen“ bedeutet hier nicht Verstehen, sondern wiederholte Optimierung anhand vieler Beispiele.



Geschichte der Computer Vision als Aufriss der Entwicklung von ML

Geschichte der Computer Vision: Überblick

- Computer Vision versucht, maschinelles Sehen zu simulieren.
- Die Geschichte ist geprägt von hohen Erwartungen, Enttäuschungen und neuen technischen Durchbrüchen.
- Wichtige Stationen reichen vom Perceptron (1958) über LeNet-5 (1998) bis zu AlexNet (2012).
- Der Aufstieg von Convolutional Neural Networks (CNNs) verändert die Bildverarbeitung grundlegend.
- Deep-Learning-Revolution in den letzten Jahren

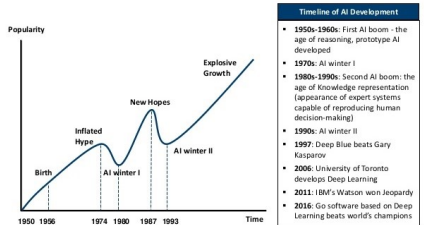
KI zwischen Hype und Ernüchterung

AI Winters

Als „AI Winter“ werden Phasen bezeichnet, in denen Erwartungen an KI-Systeme nicht erfüllt wurden und Forschungsgelder sowie öffentliches Interesse stark zurückgingen.

- Frühe KI versprach schnelle Fortschritte bei Wahrnehmung, Sprache und Denken.
- Viele Probleme erwiesen sich als deutlich schwieriger als angenommen.
- Fortschritte in Computer Vision hängen stark von Daten, Hardware und geeigneten Modellarchitekturen ab.

AI HAS A LONG HISTORY OF BEING “THE NEXT BIG THING” ...



Credit: Lim, <https://www.actuaries.digital/2018/09/05/history-of-ai-winters/>

Warum Computer Vision lange schwierig war

Das Problem

Für Menschen wirkt Sehen intuitiv. Für Computer ist ein Bild zunächst nur eine große Matrix aus Zahlenwerten.

- Objekte können unterschiedlich groß, gedreht, verdeckt oder beleuchtet sein.
- Relevante Bildmerkmale müssen erkannt werden, obwohl Pixelwerte stark variieren.
- Frühe Systeme arbeiteten oft mit manuell definierten Merkmalen.
- Moderne Systeme lernen viele Merkmale direkt aus Daten.

Zentrale Entwicklung

Der Übergang von manueller Merkmalsextraktion zu gelernten Merkmalen ist eine der wichtigsten Veränderungen in der Geschichte der Computer Vision.

Perceptron (1958)

Das Perceptron ist ein frühes Modell für maschinelles Lernen. Es lernt eine einfache Entscheidungsgrenze zwischen Klassen.

- Es zeigt früh, dass Maschinen aus Beispielen lernen können.
- Gleichzeitig ist seine Ausdruckskraft begrenzt.
- Komplexe visuelle Aufgaben erfordern mehrere Verarbeitungsschichten.
- Spätere neuronale Netze erweitern diese Idee durch tiefere Architekturen.

Convolutional Neural Networks: Grundidee

Convolution

Eine Convolution extrahiert lokale Merkmale aus verschiedenen Regionen eines Bildes. Dieselben Gewichte werden an vielen Stellen des Bildes verwendet.

- Lokale Muster wie Kanten, Kurven oder Texturen werden erkannt.
- Durch geteilte Gewichte benötigt das Modell weniger Parameter.
- Mehrere Schichten können einfache Merkmale zu komplexeren Strukturen kombinieren.
- Diese Architektur ist besonders geeignet für Bilddaten.

Convolutional Neural Networks

CNNs verbinden die Struktur von Bildern mit lernbaren Modellen und bilden damit eine Grundlage moderner Computer Vision.

LeNet-5 (1998)

LeNet-5 von LeCun, Bottou und Bengio trainiert ein sechsschichtiges Convolutional Neural Network auf graustufigen Ziffernbildern.

- **MNIST-Daten:** handgeschriebene Zeichen aus der NIST-Datenbank, ergänzt durch Verzerrungen und Verschiebungen.
- **Anwendung:** In den späten 1990er Jahren wurden bereits Schecks mit ConvNets automatisch gelesen.
- **LeNet:** zeigt, dass neuronale Netze praktische Bildklassifikationsaufgaben lösen können.

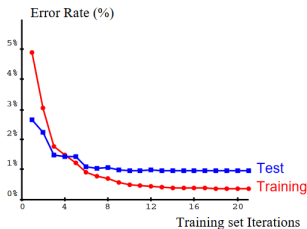
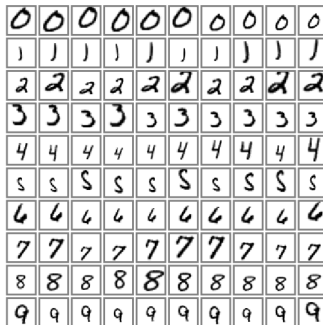


Fig. 5. Training and test error of LeNet-5 as a function of the number of passes through the 60,000 pattern training set (without distortions). The average training error is measured on-the-fly as training proceeds. This explains why the training error appears to be larger than the test error. Convergence is attained after 10 to 12 passes through the training set.



Ein wichtiger Schritt, aber noch keine Revolution

LeNet ist einflussreich, verdrängt aber frühere Ansätze der Computer Vision noch nicht vollständig.

- ConvNets sind vielseitig und können mit Verfahren der Merkmalsextraktion kombiniert werden.
- Die Trainingskosten sind hoch.
- Rechenleistung und große Trainingsdaten fehlen noch.
- Viele Anwendungen nutzen weiterhin manuell definierte Features.

Zwischenstand

Die Architektur ist vorhanden, aber die technischen Bedingungen für eine breite Deep-Learning-Revolution sind noch nicht gegeben.

Bedingung 1: Mehr Daten

Größere Trainingsdatensätze

Sinkende Speicherkosten und digitale Bildsammlungen ermöglichen deutlich größere Datensätze für Computer Vision.

MNIST: 60.000 Trainingsbilder und 10.000 Testbilder in Graustufen.

80M Tiny Images: 32x32-Bilder, mit Substantiven aus WordNet gelabelt.

Pascal VOC: Challenges mit 11.530 annotierten Bildern.

ImageNet: 14 Millionen annotierte Farbbilder.

Problem

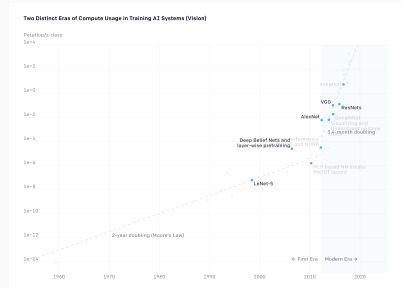
Mehr Daten bedeuten nicht automatisch bessere Daten. Verzerrungen und problematische Labels können direkt in Modelle eingehen.

Bedingung 2: Mehr Rechenleistung

GPUs

Schnelle, programmierbare GPUs ermöglichen das Training großer neuronaler Netze in praktikabler Zeit.

- Viele Bildoperationen lassen sich parallel berechnen.
- NVIDIA CUDA machte GPUs für allgemeine Berechnungen nutzbar.
- Dadurch wurden tiefere Netze und größere Datensätze praktisch trainierbar.



Computing use by neural networks over time. Credit: OpenAI,
<https://openai.com/blog/ai-and-compute/>

Deep-Learning-Frameworks

Frameworks vereinfachen die Entwicklung, das Training und die Wiederverwendung neuronaler Netze.

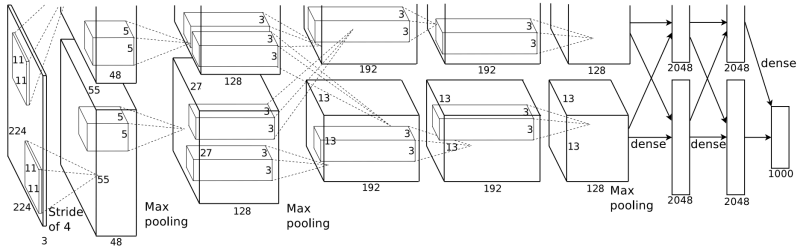
- Modelle können modular aufgebaut werden.
- Training auf GPUs wird leichter zugänglich.
- Vortrainierte Modelle und gemeinsame Benchmarks beschleunigen Forschung und Anwendung.

AlexNet (2012)

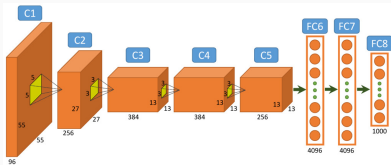
Durchbruch

AlexNet gewinnt 2012 die ImageNet Large Scale Visual Recognition Challenge mit großem Abstand.

- ImageNet umfasst Millionen von Farbbildern in 1000 Kategorien.
- AlexNet nutzt tiefe Convolutional Networks auf GPUs.
- Viele Entwicklungen der letzten Jahre bauen auf diesem Durchbruch auf.



Deep-Learning-Revolution: AlexNet



Credit: Pablo Ceres, UW-Madison

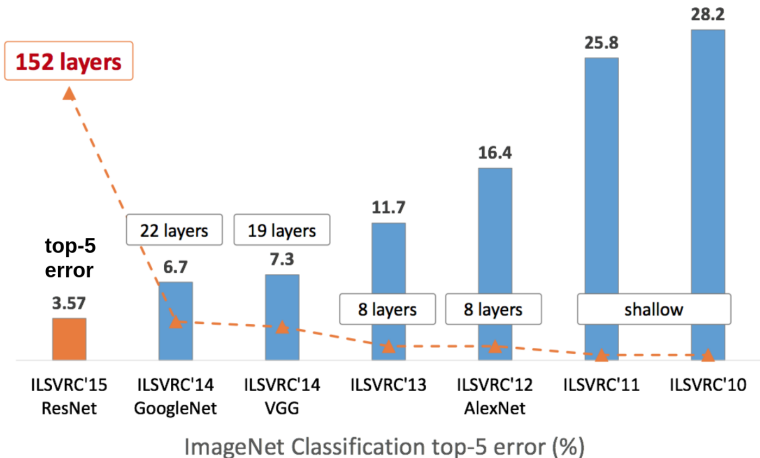
Wichtige Innovationen

- **ReLU:** beschleunigt das Training großer Netze.
- **Normalisierung:** stabilisiert das Training.
- **Dropout:** reduziert Overfitting.
- **Data Augmentation:** erzeugt zusätzliche Varianten vorhandener Trainingsbilder.

Das letzte Jahrzehnt: immer tiefere Modelle

Entwicklung

Nach AlexNet werden neuronale Netze tiefer, größer und leistungsfähiger. Bildklassifikation, Objekterkennung und Segmentierung verbessern sich deutlich.



Was bedeutet diese Geschichte für ML heute?

Drei zentrale Bedingungen

Der Erfolg moderner Computer Vision entsteht nicht durch einen einzelnen Algorithmus, sondern durch das Zusammenspiel mehrerer Faktoren.

- **Daten:** große, annotierte Trainingsdatensätze.
- **Rechenleistung:** GPUs und skalierbare Infrastruktur.
- **Architekturen:** CNNs und später weitere Deep-Learning-Modelle.
- **Software:** Frameworks, Benchmarks und wiederverwendbare Modelle.

Die Prinzipien gelten für alle Anwendungen gleichermaßen

OCR/HTR, Bildklassifikation und moderne generative Modelle beruhen auf denselben Grundfragen: Welche Daten liegen vor, was soll gelernt werden, und wie messen wir Qualität?

Generative KI fürs Schreiben?

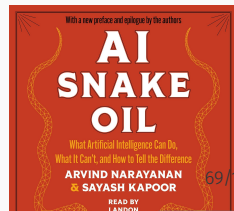
Was ist KI? Ein kurzer Überblick

Definitionen und Kontext

- sehr weiter Oberbegriff, der eine Vielzahl unterschiedlicher Technologien umfasst; generative KI ist lediglich eine davon.
- (Text-)Generative KI (GenAI) = Large Language Models (LLMs), wie die Chatbots, die wir üblicherweise verwenden.
- Starker Anstieg der Nutzung von LLMs und des öffentlichen Interesses seit 2023; wichtige Arbeiten verweisen auf ethische und ökologische Probleme (z. B. der *Stochastic Parrots*-Artikel = Bender u. a. 2021).
- Die wahrgenommene LLM-Revolution beruhte (zumindest anfangs) jedoch vor allem auf Benutzerfreundlichkeit und öffentlicher Wahrnehmung, weniger auf grundlegend neuer Technologie (Narayanan und Kapoor 2024).

Implikationen für die Wissenschaft

LLMs neigen dazu, verallgemeinernde Zusammenfassungen zu erzeugen, die spezifische Details verschleiern oder auslassen. Ihre Architektur bevorzugt zudem Informationen aus dem Mainstream gegenüber Informationen aus marginalisierten Perspektiven (Peterson 2025). Gerade die dort vorhandenen Nuancen und die Vielfalt historischer Quellen sind jedoch häufig von größerem Interesse als starke Verallgemeinerungen. Zwar können geschickte Prompts einige dieser Tendenzen abmildern, das



Was LLMs leisten: Fähigkeiten

Texte neu kombinieren = statistisch wahrscheinlichen Text auf Grundlage zuvor gesehener (teilweise aus dem Internet, teilweise aus raubkopierten Quellen stammender) Texte erzeugen.

- Das bedeutet, dass sie Texte recht gut in einen anderen Stil umschreiben können (oder ähnliche Aufgaben erfüllen). Korrekte Literaturangaben oder Fakten entstehen jedoch nur selten durch bloßes Neuarrangieren von Text.
- Selbst mathematische Beweise kommen gelegentlich zustande (durch die Umformulierung bereits existierender Beweise) und nicht dadurch, dass das Modell tatsächlich Mathematik betreibt. (Zahlen zufällig neu anzuordnen führt in der Regel nicht zu korrekter Mathematik. Einige dieser besonders peinlichen Probleme wurden inzwischen dadurch entschärft, dass im Hintergrund – für Nutzende unsichtbar – spezialisierte Werkzeuge eingebunden werden.)

Generative KI ist ein Bullshit-Generator

- ergo: **KI-generierter Text ist Bullshit** (vgl. den Essay von 1986 und das Buch von 2005 des amerikanischen Philosophen Harry G. Frankfurt, der Bullshitting als eine Form des Sprechens beschreibt, die überzeugen soll, ohne Rücksicht auf Wahrheit oder Korrektheit zu nehmen, Frankfurt 2005)
- Auch Funktionen wie „Deep Search“ verbessern dies nur begrenzt → Bezahlschranken, hochwertige Quellen blockieren KI-Crawler häufig, zudem stammt ein erheblicher Teil der verfügbaren Inhalte von unseriösen Verlagen.
- Es ist zu erwarten, dass die sogenannte „Enshittification“ (Cory Doctorow) bzw. „Slopification“ des Internets weiter zunimmt → Frühere LLMs wurden zumindest überwiegend mit von Menschen erzeugten Inhalten trainiert; inzwischen verbreiten sich jedoch zunehmend KI-generierte bzw. KI-verarbeitete („sloppifizierte“) Inhalte.

Arbeit, Vertrauen und kognitive Belastung

Verlagerung wissenschaftlicher Arbeit

Entgegen den Versprechen arbeitssparender Automatisierung verlagern LLMs wissenschaftliche Arbeit häufig, anstatt sie zu reduzieren. Anstatt Forschende von repetitiven Aufgaben zu entlasten, machen sie diese oft zu menschlichen Faktenprüfer*innen für unzuverlässige maschinelle Ausgaben. Zwar lassen sich einzelne Arbeitsschritte beschleunigen, insgesamt führt dies jedoch häufig zu einer Zunahme kognitiver und redaktioneller Arbeit.

Auch wenn es sich nur um eine kleine qualitative Studie handelt, zeigte jüngste Forschung, dass selbst sehr erfahrene Open-Source-Entwickler*innen glaubten, KI beschleunige ihre Arbeit um 20 %, obwohl sie sie tatsächlich um etwa 20 % verlangsamt (Becker u. a. 2025).

Kognitive Überlastung

Wir werden von der zusätzlichen Arbeit überwältigt, die diese Systeme erzeugen. Das notwendige Fact-Checking ist vergleichsweise anspruchsvoll und ermüdend, wodurch eine sorgfältige Prüfung zunehmend schwieriger wird – insbesondere in Kombination mit der Erwartung, KI-gestützte Arbeitsabläufe müssten schneller sein und uns dadurch Zeit verschaffen, noch mehr Arbeit zu übernehmen.

Vertrauen und Sinnzuschreibung

Wir vertrauen maschinell erzeugten Inhalten nicht nur deshalb, weil wir uns leicht von KI täuschen lassen. Wie Mark Dingemanse argumentiert (Dingemanse 2025b; Dingemanse 2025a), besitzen Menschen eine angeborene Tendenz, auch dort Sinn zu erkennen, wo keiner vorhanden ist, und diesen Sinn anschließend den Ausgaben der KI zuzuschreiben. Hinzu kommen zahlreiche weitere kognitive Verzerrungen.

Übermäßiges Vertrauen

Optimiert darauf, hilfreich zu wirken – nicht darauf, korrekt zu sein

LLMs sind strukturell davon abgehalten, Unsicherheit auszudrücken, und werden effektiv darauf trainiert, unter Unsicherheit zu raten, da dies als Teststrategie ihre Benchmark-Ergebnisse systematisch verbessert. Der Trainingsprozess belohnt selbstbewusst klingende Antworten stärker als gut kalibrierte Aussagen und erzeugt dadurch einen Dunning-Kruger-ähnlichen Effekt, bei dem das Modell übermäßig selbstsicher auftritt. Die Modelle sind darauf trainiert, nicht zu zweifeln, sondern die plausibelste Antwort auszuwählen.

Grenzen von KI-Systemen

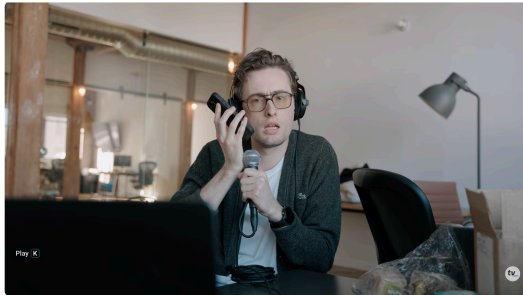
- KI ist darauf trainiert, hilfreich zu sein. Korrekte Informationen bereitzustellen ist (per Design!) nicht ihre eigentliche Aufgabe.
- Selbst die „sichereren“ Systeme sind nicht zu 100 % frei von Halluzinationen.
 - Viele Menschen nehmen fälschlicherweise an, die Nutzung von LLMs/KI entspreche dem Nachschlagen von Informationen durch eine Expertin oder einen Experten in einer Datenbank.
 - Tatsächlich handelt es sich jedoch im Wesentlichen um Statistik.
 - Eine Verankerung in überprüfbaren Fakten findet sich nur bei wenigen Werkzeugen (z. B. RAG-Systemen), und selbst dort liegt die Halluzinationsrate niemals bei null.
- LLMs können absurde semantische Fehlverknüpfungen aufweisen, etwa die Annahme, dass Menschen, die Gelb mögen, mit höherer Wahrscheinlichkeit Busfahrer*innen seien.

Paper Mills und gefälschte Publikationen

Typische Merkmale: „Tortured Phrases“

Früher enthielten gefälschte wissenschaftliche Arbeiten häufig sogenannte „tortured phrases“ (Fachbegriffe, die durch unsinnige Synonyme ersetzt wurden), etwa

- „lactose bigotry“ statt „lactose intolerance“ oder
- „kidney disappointment“ statt „kidney failure“.



I tried to buy a scientific paper



Empfohlenes Video

Christophe. *I tried to buy an academic paper*. YouTube video, accessed April 2026. März 2026. URL: <https://www.youtube.com/watch?v=SEwi0ykoXXc>

KI-Artefakte erkennen

Typische KI-Artefakte („Tells“) in Texten

- Gedankenstriche, Semikolons
- Konstruktionen wie „Es ist nicht X, sondern Y“
- Aufzählungen in Dreiergruppen



Empfohlenes Video

Lab Muffin Beauty Science. *AI slop has hit the science creators.* YouTube video, accessed April 2026. Apr. 2026.
URL: <https://youtu.be/xcq5XYkFJfY?si=9av2Zl66Us5LoEdq>

Gefährliche kognitive Verzerrungen

- **Autoritäts- und Flüssigkeitsheuristik (authority and fluency bias):** Wenn etwas flüssig und intelligent klingt, sind wir aufgrund dieser Kompetenzillusion besonders leichtgläubig.
 - Selbst Expert*innen, die es eigentlich besser wissen, vertrauen der KI oft stärker als ihrem eigenen Wissen darüber, was korrekt ist (obwohl generative KI an sich nichts „weiß“).
 - Eine höhere KI-Kompetenz kann uns paradoxerweise sogar übermäßiges Vertrauen vermitteln (und die KI-Systeme selbst sind ebenfalls übermäßig selbstsicher).
- **„Brandolinis Gesetz“** (Bullshit-Asymmetrie-Prinzip): ein Internet-Begriff, der den unverhältnismäßig hohen Aufwand beschreibt, der nötig ist, um Fehlinformationen zu widerlegen, verglichen mit der Leichtigkeit, mit der sie erzeugt werden können.

Grundregel

Verwenden Sie KI nicht zur Generierung von Inhalten, da diese mit hoher Wahrscheinlichkeit fehlerhaft oder irreführend sind. Außerdem werden Sie durch die erste Ausgabe sowie die von der KI vorgegebene Struktur beeinflusst und haben anschließend Schwierigkeiten, sich davon wieder zu lösen (wir neigen dazu, an ersten Versionen festzuhalten).

Empfehlungen

Persönliche Empfehlungen

- Nutzen Sie KI zum Anhören wissenschaftlicher Literatur (aber nicht zum Zusammenfassen).
- Nutzen Sie KI, um eigene Texte stilistisch zu verbessern, jedoch nicht, um Inhalte, Ideen oder Gliederungen zu erzeugen (verwenden Sie KI also nicht als Ausgangspunkt, um den „Anchoring Effect“ zu vermeiden).
- Wenn Sie KI verwenden, akzeptieren Sie, dass Sie zu professionellen KI-Slop-Detektor:innen werden müssen (schwierige und ausgesprochen wenig erfreuliche Aufgabe).
- Lesen Sie zu den problematischen Hintergründen von KI und ihrer Industrie, z. B. in *Atlas of AI* (Crawford 2021).

Rechtliche Empfehlungen

- Begegnen Sie KI-Ausgaben mit größtmöglicher Skepsis und prüfen Sie Fakten so sorgfältig, denn genau das ist Ihre Verantwortung; auch fast alle institutionellen Richtlinien betonen dies.
- Verwenden Sie KI nicht für personenbezogene Daten. Verstehen Sie alle Informationen, die Sie mit AI teilen als öffentlich!
- Nutzen Sie europäische KI-Angebote:
 - z.B. der GWDG: ***https://academiccloud.de/services/***: Chat AI, CoCo AI, Image AI, Voice AI
 - Nicht alle Angebote sind gleichermaßen „sicher“ – achten Sie insbesondere auf externe Dienste, die Daten beispielsweise in die USA übertragen.
- Befolgen Sie die KI-Richtlinien Ihrer Institution (ist KI erlaubt?).

Prompt Engineering

Warum Prompts wichtig sind

Grundidee

Large Language Models reagieren auf Anweisungen, die als **Prompts** bezeichnet werden. Ein Prompt ist der Text, mit dem wir beschreiben, was das Modell tun soll.

- Bessere Prompts führen häufig zu besseren Antworten.
- Prompting bedeutet, Aufgaben, Kontext und Erwartungen klar zu formulieren.
- Prompt Engineering ist im Kern klare Kommunikation mit KI-Systemen.

Analogie: Autokauf

Beim Autokauf fragt eine Verkäuferin zuerst nach Kontext:

- Wofür brauchen Sie das Auto?
- Wie viele Personen sollen mitfahren?
- Welches Budget?
- Welche Funktionalitäten sind wichtig?

Erst danach kann eine passende Empfehlung entstehen.

Prompting funktioniert ähnlich: Gute Prompts liefern den relevanten Kontext.

Zwei Grundprinzipien guter Prompts

1) Klare und spezifische Anweisungen

Ein Prompt sollte deutlich machen, was erwartet wird:

- **Aufgabe definieren:** Was soll das Modell tun?
- **Format festlegen:** Liste, Tabelle, Absatz, Code, Zusammenfassung?
- **Beispiele geben:** Welche Art von Antwort ist gemeint?
- **Grenzen setzen:** Länge, Stil, Sprache, Zielgruppe, Quellenlage.

2) Dem Modell Zeit zum Denken geben

Komplexe Aufgaben werden oft besser gelöst, wenn der Prompt einen strukturierten Bearbeitungsprozess verlangt.

- Aufgabe schrittweise lösen lassen.
- Zwischenschritte prüfen lassen.
- Fehlende Informationen identifizieren lassen.
- Erst nach Klärung eine endgültige Antwort verlangen.

Vier Kernelemente eines Prompts

Persona, Task, Context, Format

Viele Prompting-Guides empfehlen, Prompts aus wiederkehrenden Bausteinen aufzubauen:

- **Persona:** Welche Rolle soll das Modell einnehmen?
- **Task:** Welche Aufgabe soll es ausführen?
- **Context:** Welche Hintergrundinformationen sind relevant?
- **Format:** Wie soll die Ausgabe strukturiert sein?

Beispiel

Du bist Historiker:in. Fasse die Ursachen der Französischen Revolution zusammen. Schreibe in Stichpunkten für Studierende im Bachelor.

Warum das hilft

Das Modell muss weniger raten: Rolle, Ziel, Hintergrund und gewünschte Ausgabeform sind bereits festgelegt.

So sollen relevantere Teile seines 'Wissens' angesteuert werden, was die Ergebnisqualität

Weak Prompt:

"Explain the ??? Movement."

Better Prompt:

You are a historian specializing in XXX period/topic. Summarize the causes of the ??? Movement.

Constraints:

- * max 300 words
- * bullet points
- * simple language

Sprache

Oftmals funktioniert es besser auf Englisch!

Prompt Frameworks: CO-STAR und COIPO

Warum Frameworks?

Prompt Frameworks helfen, Prompts zu strukturieren.

CO-STAR

- **Context:** Hintergrund und Situation.
- **Objective:** Ziel der Aufgabe.
- **Style:** Schreib- oder Arbeitsstil.
- **Tone:** Tonfall.
- **Audience:** Zielgruppe.
- **Response:** gewünschtes Ausgabeformat.

COIPO

- **Context:** Hintergrund.
- **Objective:** Ziel.
- **Input:** Material oder Daten.
- **Process:** gewünschtes Vorgehen.
- **Output:** Ausgabeform.

Hinweis

Bei COIPO ist nur **Objective** zwingend. Die anderen Elemente werden ergänzt, wenn sie für die Aufgabe hilfreich sind.

Beispielstruktur

Context: Ich unterrichte eine Einführung in Prompt Engineering.

Objective: Erkläre die das Phänomen.

Input: Verwende die folgenden Notizen aus der Sitzung.

Process: Prüfe zuerst, ob wichtige Informationen fehlen.

Output: Gib eine gegliederte Zusammenfassung in maximal 300 Wörtern aus.

Wofür ist das nützlich?

- Die Aufgabe wird klarer.
- Das Modell erhält relevante Randbedingungen.
- Ergebnisse lassen sich leichter vergleichen und verbessern.
- Der Prompt kann in mehreren Iterationen angepasst werden.

Das Modell aktiv steuern

Gute Prompts können das Modell nicht nur zum Antworten bringen, sondern auch zum Prüfen, Nachfragen und Verbessern.

- **Begründung anfordern:** „Erkläre deine Entscheidung.“
- **Fakten prüfen lassen:** „Markiere Aussagen, die überprüft werden sollten.“
- **Lücken identifizieren lassen:** „Welche Informationen fehlen?“
- **Rückfragen verlangen:** „Stelle mir zuerst klärende Fragen.“

Beispiel

Stelle mir zuerst klärende Fragen, bevor du antwortest. Wenn Informationen fehlen, benenne sie explizit. Gib danach eine strukturierte Antwort für Bachelorstudierende.

Prompting lernen

- Learn Prompting:
<https://www.promptingguide.ai/>
- ChatGPT Prompt Engineering Course:
<https://www.deeplearning.ai/courses/chatgpt-prompt-eng>

Diese Ressourcen erklären Grundprinzipien, Beispiele und typische Strategien für verschiedene Aufgaben.

Prompt-Bibliotheken

- Awesome ChatGPT Prompts:
<https://github.com/f/prompts.chat>
- PromptHero: <https://prompthero.com/>

Prompt-Bibliotheken können als Inspirationsquelle dienen, sollten aber an Aufgabe, Kontext und Zielgruppe angepasst werden.

Gute Prompts schreiben

Zentrale Regeln

- **Kontext geben:** Hintergrund, Situation, Material.
- **Aufgabe klar definieren:** Was soll genau getan werden?
- **Format festlegen:** Liste, Tabelle, Absatz, Code, Folienstruktur.
- **Zielgruppe nennen:** Fachniveau und Vorwissen berücksichtigen.
- **Frameworks nutzen:** CO-STAR oder COIPO als Orientierung.
- **Iterieren:** Ergebnisse prüfen, nachschärfen und erneut prompten.

Beispiele: Aufgabe, Kontext und Ausgabeformat

1. Aufgabe klar formulieren

Das Modell sollte genau wissen, welche Handlung erwartet wird.

- Statt: *Erzähle etwas über Machine Learning.*
- Besser: *Erkläre die Grundlagen des Machine Learning für Einsteiger:innen in maximal 200 Wörtern.*

2. Kontext liefern

Zusätzliche Informationen helfen dem Modell, die Situation und Zielgruppe besser einzuordnen.

- Rolle oder Fachgebiet
- Vorwissen der Zielgruppe
- Anwendungsfall

3. Ausgabeformat festlegen

Definieren Sie möglichst genau, wie die Antwort aussehen soll:

- Stichpunkte oder Fließtext
- Tabelle oder JSON
- maximale Länge
- wissenschaftlicher oder einfacher Sprachstil

Beispiele, Struktur und Rollen

4. Bsps verwenden (Few-Shot Prompting)

Wenn ein bestimmtes Ausgabeformat gewünscht ist, helfen Beispiele.

- Beispiel-Eingabe
- gewünschte Beispiel-Ausgabe
- anschließend neue Eingabe

Dadurch erkennt das Modell das gewünschte Muster deutlich besser.

5. Anweisungen und Daten trennen

Eine klare Struktur erleichtert dem Modell die Verarbeitung.

- Aufgabe
- Eingabedaten
- gewünschtes Ausgabeformat

Abschnittsüberschriften oder Trenner (z. B. `### Text`) erhöhen die Übersichtlichkeit.

6. Eine Rolle vergeben

Beispielsweise:

- „Du bist Historiker:in.“
- „Du bist Data Scientist.“
- „Du bist technische Redakteurin.“

Die Rolle beeinflusst Stil, Detailgrad und fachliche Perspektive.

```
Convert sentences to JSON.  
Example:  
Input: Älice lives in Berlin.  
"  
Output:  
{ "name": "Älice", "city": "Berlin" }  
Now convert:  
Input: "John lives in Paris."
```

Eine universelle Prompt-Vorlage

Prompt-Template

Role: Welche Rolle soll das Modell übernehmen?

Task: Welche Aufgabe soll gelöst werden?

Context: Welche Hintergrundinformationen sind wichtig?

Instructions:

- Regel 1
- Regel 2
- Regel 3

Output Format: Beschreibung der gewünschten Ausgabe.

Beispiel

Role: Technische:r Redakteur:in

Task: Docker für Einsteiger erklären

Context: Grundkenntnisse Linux vorhanden

Output:

- fünf Stichpunkte
- einfache Sprache
- ein Praxisbeispiel

Warum Templates?

Sie sorgen dafür, dass wichtige Informationen nicht vergessen werden und erleichtern die Wiederverwendung von Prompts.

Typische Bausteine

- Rolle
- Aufgabe
- Kontext
- Regeln
- Ausgabeformat

Fortgeschrittene Prompting-Techniken I

Chain-of-Thought

Das Modell wird aufgefordert, eine Aufgabe schrittweise zu lösen.

- nützlich für Rechen- und Schlussfolgerungsaufgaben
- zerlegt komplexe Probleme in Zwischenschritte

Löse das Problem Schritt für Schritt.

Few-Shot Prompting

Dem Modell werden Beispiele vor der eigentlichen Aufgabe gegeben.

- hilfreich für strukturierte Ausgaben
- Modell imitiert das gezeigte Muster

Input: Alice lives in Berlin

Output:

{"name:Älice", "city:"Berlin"}

Zero-Shot Prompting

Das Modell erhält nur eine Anweisung, aber keine Beispiele.

- geeignet für einfache Aufgaben
- schnell und direkt

Fasse diesen Artikel in drei Stichpunkten zusammen.

Self-Consistency

Das Modell entwickelt mehrere Lösungswege und wählt die konsistenteste Antwort.

- verbessert Zuverlässigkeit bei komplexen Aufgaben
- reduziert Zufallsfehler einzelner Lösungswege

Fortgeschrittene Prompting-Techniken II

Tree-of-Thought

Das Modell soll mehrere mögliche Lösungszweige entwickeln, bewerten und auswählen.

- nützlich für Planung
- hilfreich bei Entscheidungen mit Alternativen

Erzeuge mehrere Lösungen, bewerte sie und wähle die beste.

ReAct: Reason + Act

Das Modell kombiniert Schlussfolgern mit Aktionen, z. B. Tool-Nutzung.

- geeignet für Recherche oder Berechnungen
- verbindet Denken, Aktion und Beobachtung

Thought → Action → Observation → Answer

Self-Ask

Das Modell formuliert zuerst Teilfragen, bevor es antwortet.

- gut für Forschung und komplexe Fragen
- macht fehlende Zwischenschritte sichtbar

Formuliere zuerst die nötigen Teilfragen.

Step-Back Prompting

Das Modell erklärt zuerst das allgemeine Prinzip und löst dann den konkreten Fall.

- verbessert Einordnung
- verhindert zu schnelles Antworten

Rollen und Meta-Prompting

Role Prompting

Eine Rolle steuert Perspektive, Detailgrad und Stil.

- *Du bist Cybersecurity-Expert:in.*
- *Du bist Historiker:in.*
- *Du bist technische:r Redakteur:in.*

Meta-Prompting

Das Modell wird genutzt, um bessere Prompts zu entwerfen.

- hilfreich bei unklaren Aufgaben
- geeignet zur Iteration
- kann mehrere Prompt-Versionen vergleichen

Erstelle den besten Prompt, um Quantencomputing für Einsteiger:innen zu erklären.

Fünf Kernbestandteile

Fast alle guten Prompts enthalten:

1. **Task:** Was soll getan werden?
2. **Context:** Welche Informationen sind wichtig?
3. **Constraints:** Welche Regeln gelten?
4. **Output format:** Wie soll die Antwort aussehen?
5. **Examples:** Welche Muster sollen initiiert werden?

Formel

**Prompt = Role + Task + Context +
Constraints + Output Format + Examples**

Häufige Fehler beim Prompting

Zu vage formulieren

Schwach: *Erkläre KI.*

Besser: *Erkläre neuronale Netze für Einsteiger:innen in 150 Wörtern.*

Keinen Kontext geben

Schwach: *Fasse das zusammen.*

Besser: *Fasse diesen Forschungsartikel für ein wirtschaftliches Publikum zusammen.*

Kein Ausgabeformat festlegen

Schwach: *Nenne gute Programmiersprachen.*

Besser: *Erstelle eine Tabelle mit Sprache, Anwendungsfall und Schwierigkeit.*

Zu viele Aufgaben in einem Prompt

Schwach: *Erkläre KI, schreibe Code, vergleiche Modelle und erstelle Folien.*

Besser: Aufgaben in mehrere Prompts aufteilen.

Keine Beispiele geben

Bei strukturierten Aufgaben verbessern Beispiele die Konsistenz der Ausgabe.

Token-Limits ignorieren

Zu viel Text kann Kontextfenster überladen, Eingaben abschneiden oder Antworten verkürzen.

Super Prompt Pattern

Muster

Du bist Expert:in für

Feld

.

Deine Aufgabe:

klare Zielsetzung

Kontext:

Hintergrund

Schritte:

1. Verstehe das Problem.
2. Denke die Lösung schrittweise durch.
3. Gib die Lösung aus.

Constraints:

Länge, Ton, Stil

Output format:

strukturiertes Format

Wann nützlich?

- bei komplexen Aufgaben
- bei fachlichen Erklärungen
- bei Planung und Entscheidung
- bei Aufgaben mit festem Ausgabeformat

Kernidee

Das Muster verbindet Rolle, Aufgabe, Kontext, Arbeitsprozess und Ausgabeformat.

Pro-Tipp: Interaktives Prompting

Statt direkt zu fragen

Schreibe einen Blogpost über KI.

Besser

Stelle mir zuerst Fragen, um alle Informationen zu sammeln, die du brauchst, um einen Blogpost über KI zu schreiben.

Warum das hilft

- Das Modell muss weniger raten.
- Zielgruppe, Ton, Länge und Inhalt werden vorab geklärt.
- Die Antwort passt besser zur tatsächlichen Aufgabe.

Ein-Satz-Zusammenfassung

Ein guter Prompt ist klar, strukturiert, kontextreich und begrenzt.

Merksatz

Wenn die Aufgabe unklar ist, sollte der erste Prompt nicht die Lösung verlangen, sondern gute Rückfragen.

Universelles Prompt-Template

Grundstruktur

Role: Du bist [Rolle].

Task: [Was soll das Modell tun?]

Context: [Hintergrundinformationen]

Instructions:

- Regel 1
- Regel 2
- Regel 3

Constraints:

- Länge
- Ton
- Format

Output format: [gewünschte Ausgabeform]

Beispiel

Role: Senior Software Engineer

Task: Erkläre Docker für Einsteiger:innen.

Context: Zielgruppe kennt Linux, aber keine Container.

Instructions:

- einfache Sprache
- ein Alltagsbeispiel
- keinen unnötigen Jargon

Output: fünf Stichpunkte.

Warum das funktioniert

Klare Aufgabe, Kontext und Ausgabeformat reduzieren Mehrdeutigkeit.

Fortgeschrittene Prompting-Techniken I

Chain-of-Thought

Das Modell wird aufgefordert, eine Aufgabe schrittweise zu lösen.

- nützlich für Rechen- und Schlussfolgerungsaufgaben
- zerlegt komplexe Probleme in Zwischenschritte

Löse das Problem Schritt für Schritt.

Few-Shot Prompting

Dem Modell werden Beispiele vor der eigentlichen Aufgabe gegeben.

- hilfreich für strukturierte Ausgaben
- Modell imitiert das gezeigte Muster

Input: Alice lives in Berlin

Output:

{"name":"Alice", "city":"Berlin"}

Zero-Shot Prompting

Das Modell erhält nur eine Anweisung, aber keine Beispiele.

- geeignet für einfache Aufgaben
- schnell und direkt

Fasse diesen Artikel in drei Stichpunkten zusammen.

Self-Consistency

Das Modell entwickelt mehrere Lösungswege und wählt die konsistenteste Antwort.

- verbessert Zuverlässigkeit bei komplexen Aufgaben
- reduziert Zufallsfehler einzelner Lösungswege

Fortgeschrittene Prompting-Techniken II

Tree-of-Thought

Das Modell soll mehrere mögliche Lösungszweige entwickeln, bewerten und auswählen.

- nützlich für Planung
- hilfreich bei Entscheidungen mit Alternativen

Erzeuge mehrere Lösungen, bewerte sie und wähle die beste.

ReAct: Reason + Act

Das Modell kombiniert Schlussfolgern mit Aktionen, z. B. Tool-Nutzung.

- geeignet für Recherche oder Berechnungen
- verbindet Denken, Aktion und Beobachtung

Thought → Action → Observation → Answer

Self-Ask

Das Modell formuliert zuerst Teilfragen, bevor es antwortet.

- gut für Forschung und komplexe Fragen
- macht fehlende Zwischenschritte sichtbar

Formuliere zuerst die nötigen Teilfragen.

Step-Back Prompting

Das Modell erklärt zuerst das allgemeine Prinzip und löst dann den konkreten Fall.

- verbessert Einordnung
- verhindert zu schnelles Antworten

Rollen und Meta-Prompting

Role Prompting

Eine Rolle steuert Perspektive, Detailgrad und Stil.

- *Du bist Cybersecurity-Expert:in.*
- *Du bist Historiker:in.*
- *Du bist technische:r Redakteur:in.*

Meta-Prompting

Das Modell wird genutzt, um bessere Prompts zu entwerfen.

- hilfreich bei unklaren Aufgaben
- geeignet zur Iteration
- kann mehrere Prompt-Versionen vergleichen

Erstelle den besten Prompt, um Quantencomputing für Einsteiger:innen zu erklären.

Fünf Kernbestandteile

Fast alle guten Prompts enthalten:

1. **Task:** Was soll getan werden?
2. **Context:** Welche Informationen sind wichtig?
3. **Constraints:** Welche Regeln gelten?
4. **Output format:** Wie soll die Antwort aussehen?
5. **Examples:** Welche Muster sollen initiiert werden?

Formel

**Prompt = Role + Task + Context +
Constraints + Output Format + Examples**

Häufige Fehler beim Prompting

Zu vage formulieren

Schwach: *Erkläre KI.*

Besser: *Erkläre neuronale Netze für Einsteiger:innen in 150 Wörtern.*

Keinen Kontext geben

Schwach: *Fasse das zusammen.*

Besser: *Fasse diesen Forschungsartikel für ein wirtschaftliches Publikum zusammen.*

Kein Ausgabeformat festlegen

Schwach: *Nenne gute Programmiersprachen.*

Besser: *Erstelle eine Tabelle mit Sprache, Anwendungsfall und Schwierigkeit.*

Zu viele Aufgaben in einem Prompt

Schwach: *Erkläre KI, schreibe Code, vergleiche Modelle und erstelle Folien.*

Besser: Aufgaben in mehrere Prompts aufteilen.

Keine Beispiele geben

Bei strukturierten Aufgaben verbessern Beispiele die Konsistenz der Ausgabe.

Token-Limits ignorieren

Zu viel Text kann Kontextfenster überladen, Eingaben abschneiden oder Antworten verkürzen.

Super Prompt Pattern

Muster

Du bist Expert:in für

Feld

.

Deine Aufgabe:

klare Zielsetzung

Kontext:

Hintergrund

Schritte:

1. Verstehe das Problem.
2. Denke die Lösung schrittweise durch.
3. Gib die Lösung aus.

Constraints:

Länge, Ton, Stil

Output format:

strukturiertes Format

Wann nützlich?

- bei komplexen Aufgaben
- bei fachlichen Erklärungen
- bei Planung und Entscheidung
- bei Aufgaben mit festem Ausgabeformat

Kernidee

Das Muster verbindet Rolle, Aufgabe, Kontext, Arbeitsprozess und Ausgabeformat.

Pro-Tipp: Interaktives Prompting

Statt direkt zu fragen

Schreibe einen Blogpost über KI.

Besser

Stelle mir zuerst Fragen, um alle Informationen zu sammeln, die du brauchst, um einen Blogpost über KI zu schreiben.

Warum das hilft

- Das Modell muss weniger raten.
- Zielgruppe, Ton, Länge und Inhalt werden vorab geklärt.
- Die Antwort passt besser zur tatsächlichen Aufgabe.

Ein-Satz-Zusammenfassung

Ein guter Prompt ist klar, strukturiert, kontextreich und begrenzt.

Merksatz

Wenn die Aufgabe unklar ist, sollte der erste Prompt nicht die Lösung verlangen, sondern gute Rückfragen.

Literatur

- [1] Joel Becker u. a. ***Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity.*** arXiv preprint. 2025. DOI: [10.48550/arXiv.2507.09089](https://doi.org/10.48550/arXiv.2507.09089). arXiv: [2507.09089](https://arxiv.org/abs/2507.09089). URL: <https://doi.org/10.48550/arXiv.2507.09089>.
- [2] Emily M. Bender u. a. „On the Dangers of Stochastic Parrots: Can Language Models Be Too Big?“ In: *Proceedings of the 2021 ACM Conference on Fairness, Accountability, and Transparency*. FAccT '21. Virtual Event, Canada: Association for Computing Machinery, 2021, S. 610–623. ISBN: 9781450383097. DOI: [10.1145/3442188.3445922](https://doi.org/10.1145/3442188.3445922). URL: <https://doi.org/10.1145/3442188.3445922>.
- [3] Christophe. ***I tried to buy an academic paper.*** YouTube video, accessed April 2026. März 2026. URL: <https://www.youtube.com/watch?v=SEwiOykoXXc>.
- [4] Kate Crawford. ***Atlas of AI. Power, Politics, and the Planetary Costs of Artificial Intelligence.*** New Haven: Yale University Press, 2021.
- [5] Mark Dingemanse. ***Eight Books I Built On in My Lecture ...*** Mastodon post, March 6. März 2025. URL: <https://scholar.social/@dingemansemark/114117116366459913>.

- [6] Mark Dingemanse. *Why Are LLMs So Irresistible?* Keynote presented at DHD 2025, Bielefeld, March 4. 2025. URL: https://dhd2025.dig-hum.de/?page_id=532.
- [7] Harry G. Frankfurt. *On Bullshit*. Princeton, New Jersey: Princeton University Press, 2005.
- [8] Lab Muffin Beauty Science. *AI slop has hit the science creators*. YouTube video, accessed April 2026. Apr. 2026. URL: <https://youtu.be/xcq5XYkFJfY?si=9av2Zl66Us5LoEdq>.
- [9] Arvind Narayanan und Sayash Kapoor. *AI Snake Oil: What Artificial Intelligence Can Do, What It Can't, and How to Tell the Difference*. Princeton, NJ: Princeton University Press, 2024.
- [10] AJ Peterson. „AI and the problem of knowledge collapse“. In: *AI & Society* 40 (2025), S. 3249–3269. DOI: [10.1007/s00146-024-02173-x](https://doi.org/10.1007/s00146-024-02173-x).
- [11] Grant Sanderson und Josh Pullen. *Neural Networks. Chapter 1. But What Is a Neural Network?* Lesson by Grant Sanderson; text adaptation by Josh Pullen. 3Blue1Brown. 5. Okt. 2017. URL: <https://www.3blue1brown.com/lessons/neural-networks/> (besucht am 06.08.2026).

- [12] Grant Sanderson und Josh Pullen. ***Neural Networks. Chapter 2. Gradient Descent, How Neural Networks Learn.*** Lesson by Grant Sanderson; text adaptation by Josh Pullen. 3Blue1Brown. 16. Okt. 2017. URL: <https://www.3blue1brown.com/lessons/gradient-descent/> (besucht am 06.08.2026).